

Governance as Inference-Time Capability Substitution: Reducing Tokens per Correct Software-Engineering Task with Deterministic Requirements and Verification

Layer 1 Labs

<https://github.com/layer1labs/specsmith>

Preprint — July 2026

Abstract

Language-model comparisons usually hold the agent scaffold fixed and vary the model. We study the complementary systems question: can deterministic governance compensate for measured model capability in repository-level software tasks? We introduce GovernanceBench, an executable benchmark and matched 2×2 design in which a lower-tier and frontier model each run with an ungoverned agent loop and with SPECSMITH FULL governance. The controller exposes a stable five-tool interface, retrieves only requirement-linked context, enforces visible tests without exposing hidden acceptance oracles, and audits every trace. Our primary efficiency measure is tokens per correct answer; correctness is a hard non-inferiority constraint. In the preregistered eight-task, ten-repetition experiment, GPT-5.6 Terra with FULL governance passed 80/80 cells at 11,737 tokens per correct answer, while GPT-5.6 Sol in the raw loop passed 65/80 at 28,023. The TPCA ratio was 0.419 (fixed-suite 95% interval 0.358–0.485; task-cluster interval 0.212–0.708). Both preregistered mixed-suite substitution gates passed. In a prespecified coding-only sensitivity, Terra FULL passed 60/60 at 15,649 TPCA versus Sol raw 55/60 at 30,250, but its correctness lower bound was -5.7 percentage points, just outside the -5 -point non-inferiority margin. Published 8B and mini-model routes failed admission. The evidence supports task-conditional substitution for the exact API routes and benchmark distribution, not universal model equivalence or a claim that small models replace frontier models.

1 Introduction

Repository-level coding agents must localize requirements, edit multiple files, run tools, interpret failures, and decide when evidence is sufficient. Benchmarks such as SWE-bench expose this interaction complexity [5], while SWE-agent shows that the agent-computer interface itself can materially change performance [15]. Consequently, “which model is best?” is incomplete: practical outcomes depend on the joint system of model, context policy, tools,

validators, and stopping rules.

This paper asks whether a governance controller can act as *inference-time capability substitution*. The target is not to make a lower-tier route intrinsically equivalent to a frontier model. It is to determine whether a lower-tier model embedded in a deterministic, evidence-seeking controller can preserve task correctness while consuming fewer tokens or dollars than a frontier model in an ungoverned loop.

The question connects three literatures. Interactive reasoning methods ground generation in external actions and feedback [7, 10, 16]. Simpler software-agent pipelines can outperform more elaborate loops [14]. Model cascades and routers exploit heterogeneous cost–quality tradeoffs [2, 8]. SPECSMITH differs by routing not only between models but between probabilistic generation and deterministic governance operations: requirement lookup, scoped retrieval, test execution, safety refusal, evidence compression, and stopping.

We make four contributions:

1. a matched 2×2 experimental design separating model capability from governance lift;
2. a primary metric, tokens per correct answer, coupled to a preregistered correctness non-inferiority gate;
3. deterministic fixed-suite and task-cluster uncertainty analysis plus machine-readable claim gates; and
4. a versioned compact evidence artifact containing per-cell outcomes, workflow provenance, negative results, and a post-run weakness audit, backed by versioned tasks, validators, and hidden acceptance oracles.

2 Related Work

Executable software-engineering benchmarks. SWE-bench evaluates issue resolution against executable repository tests [5]. Static benchmark contamination and memorization are material threats [6]; SWE-bench-Live addresses freshness and repository diversity with continuously updated tasks and containerized environments [17].

GovernanceBench is smaller and synthetic by design. It cannot estimate population-level coding ability, but its fixtures make controller ablation, exact counterfactual matching, and trace-level token accounting practical.

Agent interfaces and feedback. SWE-agent attributes substantial gains to a model-oriented interface [15]. ReAct interleaves reasoning with environmental actions [16]; Reflexion and Self-Refine use iterative feedback without parameter updates [7, 10]. Agentless demonstrates that constrained localization, repair, and validation can be competitive at low cost [14]. These findings motivate a compact tool surface and controller-owned validation rather than a large catalogue of agent skills.

Cost-aware model selection. FrugalGPT and RouteLLM demonstrate that heterogeneous models can be cascaded or routed to improve cost-quality tradeoffs [2, 8]. Our unit of analysis is a complete software task, and our router signal is not a learned preference score. Admission and post-run audit use executable correctness, requirements, scope boundaries, token receipts, and task-specific historical envelopes.

Reproducibility and variance. Benchmark point estimates can be dominated by uncontrolled variation [1]. Recent analysis of 640 LLM-for-software-engineering papers reports persistent artifact, environment, and versioning gaps [11]. We therefore repeat every release cell, pin task and controller revisions to one commit, reject missing provider cells, record model routes and tool-schema hashes, and publish interval estimates rather than relying on means alone.

3 System

3.1 Governance controller

SPECSMITH FULL wraps a language model with six mechanisms:

1. **Requirement contracts:** accepted requirements, linked tests, scope, risk, and epistemic constraints are loaded before edits.
2. **Bounded context:** versioned requirement-linked files may be preloaded; all other content is retrieved just in time.
3. **Stable interface:** every model turn sees the same five tools: batched and scalar read/write operations plus completion.
4. **Deterministic gates:** ambiguous or destructive tasks can stop without an LLM call; coding completion requires current public validation evidence.

5. **Evidence compression:** unchanged reads become digest receipts, superseded observations are removed, and only active milestone state remains in long-horizon context.
6. **Hidden scoring:** acceptance oracles run only after the model stops and never supply repair hints.

The design intentionally excludes broad repository preloads, mutable tool schemas, evaluator leakage, and unbounded self-reflection. A deterministic post-run audit identifies correctness, scope, retrieval, repair, tool-schema, and cost weaknesses and selects the next admissible experiment.

3.2 Promotion funnel

New model routes receive one long-horizon governed admission. Controller changes receive one cell on each affected path. Successful candidates advance to a four-task, five-repetition 2×2 screen. Only then may the complete eight-task, ten-repetition substitution profile run. This funnel prevents a failed candidate from becoming persuasive merely through additional spend.

4 GovernanceBench

4.1 Tasks and conditions

The release grid contains eight tasks: two feature/bug tasks, two deterministic clarification or safety gates, three cross-file feature/refactor tasks, and one long-horizon polyglot incident console spanning Python, Go, TypeScript/React, JSON Schema, browser tests, and documentation. Six tasks require code changes; two assess whether the system declines or clarifies unsafe or under-specified work.

The substitution experiment crosses two models with two conditions:

Cell	Purpose
Lower-tier raw	lower-tier capability baseline
Lower-tier FULL	candidate governed system
Frontier raw	substitution target
Frontier FULL	frontier governance control

All four cells use identical task fixtures, repetition identifiers, provider APIs, temperature policy, turn cap, completion cap, and hidden oracles. Provider errors invalidate a cell rather than counting as free failures.

4.2 Metrics

For runs $i = 1, \dots, n$, with token count t_i , cost c_i , and correctness $y_i \in \{0, 1\}$, we report

$$\text{TPCA} = \frac{\sum_i t_i}{\sum_i y_i}, \quad \text{CoP} = \frac{\sum_i c_i}{\sum_i y_i}. \quad (1)$$

This formulation charges failed attempts to the expected cost of obtaining a correct answer. Raw tokens are primary because provider prices and cache discounts change; estimated dollars are secondary.

Correctness is evaluated by deterministic lint, public tests, task-specific contract validators, and an evaluator-isolated acceptance oracle. We also record turns, wall time, files read and written, scope expansion, first-pass rate, provider-reported cache tokens, and a stable tool-schema hash.

4.3 Hypotheses and claim gates

The release profile and thresholds were committed at revision 75a8c79 before workflow dispatch. The primary comparison is lower-tier FULL minus frontier raw:

H1: correctness non-inferiority. The lower 95% bound on the pass-rate difference is at least -5 percentage points.

H2: token superiority. The upper 95% bound on the TPCA ratio is below 1.

H3: cross-task robustness. H1 and H2 also hold when task identifiers, not only repetitions, are resampled.

H4: governance lift. Lower-tier FULL improves correctness or TPCA relative to the same lower-tier raw model.

We use Wilson score bounds for binomial rates [13] and a deterministically seeded, 10,000-sample hierarchical bootstrap [3]. The fixed-suite analysis resamples repetitions within each task. The task-cluster analysis additionally resamples task IDs; it tests robustness to resampling the benchmark tasks but cannot establish performance on fresh repositories. A result can therefore support fixed-suite substitution while failing cross-task substitution. No result supports universal model equivalence.

5 Results

Preregistered release inference. The lower-tier FULL/frontier-raw TPCA point ratio was 0.419. Its fixed-suite 95% interval was 0.358–0.485; the task-cluster interval was 0.212–0.708. The corresponding correctness-difference intervals were 3.3–34.1 pp and -4.4 –54.7 pp. The machine-readable release gate passed. Fixed-suite substitution was supported, and cross-task substitution was supported.

Coding-only sensitivity. After excluding deterministic clarification and safety gates, lower-tier FULL passed 60/60 coding cells at 15,649 TPCA; frontier raw passed 55/60 at 30,250 TPCA. The TPCA ratio was 0.517, with fixed-suite 95% interval 0.437–0.608 and task-cluster interval 0.328–0.870. The correctness-difference intervals

were -5.7 –26.1 pp and -6.0 –31.8 pp. Coding-only fixed-suite substitution was not supported; this sensitivity analysis was specified for interpretation after the mixed-suite release gate and is not relabelled as the primary endpoint.

The primary mixed-suite point estimate corresponds to 58.1% lower TPCA and 60.1% lower estimated cost per correct answer for Terra FULL than Sol raw. The task-cluster correctness lower bound was -4.4 percentage points, clearing the preregistered -5 -point margin by 0.6 points. The coding-only sensitivity went in the same observed direction but did not clear that margin; deterministic clarification and safety gates therefore matter to the strength of the primary claim. We report both endpoints rather than relabelling the sensitivity after observing the result.

The four-task screen demonstrates why both uncertainty views are needed. Its point TPCA ratio is 0.653. The fixed-suite 95% interval is 0.539–0.784, while the task-cluster interval is 0.339–1.396. Terra FULL is less token-efficient than Sol raw on T1 and T10 but substantially more efficient on T13 and T28. The aggregate is therefore useful routing evidence, not a universal replacement result.

6 Trace-Guided Ablations

The benchmark audit converts failures and expensive correct runs into bounded controller experiments. Three examples illustrate the loop:

- A T10 controller denied reads of an imported model and dependency manifest. Adding exactly those files reduced one admission from 63,128 tokens and 11 turns to 9,305 tokens and two turns.
- Repeated T11 first actions justified a bounded preload, reducing a diagnostic from 23,472 tokens and six turns to 11,210 tokens and two turns.
- Applying the same idea to T13 increased tokens from a 14,424-token broad-run mean to 16,190 in admission. The preload was removed.

The negative T13 result is important: more context is not monotonically beneficial, and governance optimization itself requires admission controls.

Open-model admissions provide a second negative control. Qwen3.6-35B-A3B completed the long-horizon task with a stable tool schema but used 67,701 tokens, 3.87 times the governed frontier anchor. It was not promoted to repeated spend. Correctness alone is insufficient for an efficiency claim.

A published-size Llama 3.1 8B route failed three governed T28 admissions. Strict recovery of its text-serialized function calls removed a serving protocol ambiguity, but the final route still issued 20 actions serially,

Table 1: Preregistered eight-task, four-cell, n=10 substitution replication. Workflow 30210886840, commit 75a8c7911187. TPCA is tokens per correct answer and CoP is estimated API cost per passing task.

System	Correct	TPCA	CoP (USD)
gpt-5.6-terra raw	56/80	45,915	0.1642
gpt-5.6-terra + Specssmith FULL	80/80	11,737	0.0602
gpt-5.6-sol raw	65/80	28,023	0.1511
gpt-5.6-sol + Specssmith FULL	80/80	8,082	0.0752

exhausted the turn cap, and failed at 98,543 tokens. GPT-4o mini also failed T28 at 107,896 tokens after focused-boundary repair. These admissions block the stronger marketing claim that an 8B or mini model replaces a frontier model. They do not negate the separately tested lower-tier Terra system, whose parameter count and architecture are not inferred.

A later reasoning-capable cohort covered exact managed routes from 20B to 32B. GPT-OSS-20B, Qwen3-Coder-30B-A3B, GLM-4.7-Flash, and Qwen3-32B failed their governed T28 admissions. Qwen3.6-27B initially stopped after narrating its next milestone; a bounded controller repair then produced a correct public-and-hidden cell at 72,255 tokens and 14 model turns. This establishes n=1 capability for the published 27B route, but not efficient substitution: the cell used 4.13 times the 17,501.7-token governed Sol envelope and was not promoted to repeated spend. The combined evidence separates “can finish once” from “replaces a frontier system efficiently.”

A subsequent exact-boundary confirmation also passed at 77,776 tokens and 14 turns. It is not pooled with the first cell because the controller commit changed between runs. Its higher 4.44-times frontier-envelope ratio shows that 27B capability can recur while preserving the negative efficiency conclusion.

A subsequent controller-protocol isolation produced a material within-route improvement. Automatic scalar-parallel file calls completed the same T28 public and hidden gates in six turns and 26,850 tokens, 62.8 percent below the original correct admission. The cell is still an n=1 diagnostic and consumes 1.53 times the 17,501.7-token governed Sol envelope. Required-tool, write-only, context-compaction, and independent-validator-authority variants did not beat it. The result therefore supports editing-interface optimization, not a frontier-replacement claim.

A later native-interface series isolated provider-visible editing structure. Managed Novita routes for Qwen3-Coder-Next and Qwen3-Coder-480B failed the same task. A fixed-scalar milestone bundle restored Qwen3.6-27B correctness at 71,090 tokens and 13 turns, but consumed 4.06 times the governed Sol envelope and 2.65 times the scalar-parallel winner. Exact single-hunk edits remained near 27,000 tokens but could not express a coupled import-and-annotation repair atomically. The resulting bounded three-hunk patch interface passed local safety and routing tests; its live runtime was censored by a provider 504, and the single retry failed route avail-

ability before execution. Both censored attempts are excluded from correctness and efficiency inference. These results show that tool-serving structure can alter correctness while leaving the broad frontier-replacement claim unsupported.

A follow-up used the same atomic interface with the hosted Qwen3-Coder-30B-A3B route after an exact structured-tool probe. The baseline failed at 123,384 tokens, 20 turns, and four of ten files. A scoped variant removed model-owned rereads and reduced the failed attempt to 34,892 tokens, 10 turns, and 29.86 seconds—reductions of 71.7 percent in tokens, 73.9 percent in estimated cost, and 78.3 percent in wall time. It still failed the independent oracle, so this is a failure-cost result rather than a TPCA or substitution result. Requiring a tool on every turn regressed to 81,648 tokens and repeated one three-action batch across eleven turns. We reject required tool choice and add a content-free batch-repeat guard that would stop the observed sequence after its fourth identical batch.

We separately preregistered an ephemeral endpoint for the official FP8 checkpoint, vLLM 0.24.0, and the native `qwen3_xml` parser [4, 9, 12]. The first attempt was censored before provisioning because the credential lacked endpoint-write permission. After correcting that permission, two endpoint runs passed an exact required-tool probe and confirmed pause and deletion. Atomic and scoped automatic-tool cells still failed at 34,146 and 53,451 tokens. A scoped required-tool cell failed at 181,884 tokens and 20 turns after changing the failure mode from premature narration to completion truncation and milestone fragmentation. The two endpoint receipts report approximately \$0.430977 combined infrastructure cost. Thus native parser compatibility removes a serving-stack ambiguity but is insufficient for correctness, and globally required tool choice is rejected for this model and task.

We then tested the newer Qwen3.6-35B-A3B-FP8 checkpoint on one A100 with vLLM 0.24.0, a 65,536-token context, the native `qwen3.coder` tool parser, and the `qwen3` reasoning parser. Both cells passed an exact tool probe and recorded successful endpoint pause and deletion. Generic required-tool recovery failed T29 after one of four milestones, 30,287 tokens, a length stop, and two empty continuations. Exact named-tool recovery advanced through three milestones but failed at 105,087 tokens and the 20-turn cap. Deterministic sanitation

removed 96 duplicate actions, and bounded malformed-patch recovery still left Go dependency and TypeScript query repairs incomplete. The latter cell consumed 5.97 times the correct Terra T29 v7 token anchor without a correct answer. This valid native-route failure rejects repetition and shows that parser compatibility plus governance recovery did not substitute for model capability on the fresh polyglot task.

The same controls were then exercised through the native Responses tool route of the stronger GPT-5.6 Sol system. Replacing eight fixed nullable file slots with one bounded strict array of file objects preserved all-or-nothing milestone writes while reducing provider-visible schema overhead. An independent $n = 10$ T28 confirmation passed public and hidden validation in all ten rows, completed in five model turns with 100 percent first-pass success, and averaged 17,907 tokens per correct answer (1.77 percent coefficient of variation). This was 7.2 percent below the preceding stable-schema native controller, but 2.3 percent above the 17,502-token release anchor. The result supports native structured editing as a material controller optimization on this task and route; it does not establish a universal model or task-family advantage.

We then froze v4 and changed only the model route to GPT-5.6 Terra. After a visible-validator repair, T28 passed five of five screening rows, but its independent ten-row confirmation passed only nine. The failed row repeatedly targeted the first milestone until the bounded loop guard stopped execution, leaving three product boundaries incomplete. The sample consumed 176,832 tokens and yielded 19,648 tokens per correct answer, so the audit rejected promotion. We next introduced T29, an independent synthetic release-control repository with a different domain, seven-field contract, starter, validators, and hidden oracle while preserving the four milestone sizes and frozen controller. T29 passed five of five rows at 22,038 mean tokens per correct answer and \$0.1299 mean estimated cost, but every row entered App empty-state repair and one invoked loop recovery. Re-audit showed the original CSS-hotspot label was a parser artifact from a relative import in supplied App content. The public validator required the literal word “empty” even when the hidden oracle accepted the structural zero-length/no-release state.

A trace-derived v5 package aligned that check, added CSS-only milestone-three validation for focus, disabled actions, and responsive layout, and bounded unchanged repair streaks at two. Admission workflow 30453721376 passed first-pass at 17,407 tokens. The earned five-row workflow 30454018932 passed five of five at 18,055 mean tokens per correct answer, \$0.1199 mean estimated cost, 5.2 turns, 80% first-pass, and zero loop recoveries. One row repaired a missing Playwright visibility assertion in one patch. Relative to v4, mean tokens fell 18.1%, cost 7.7%, and turns 18.8%; because the task/validator contract and recovery controller changed together, this is a package-level delta rather than a single-factor causal estimate. The admitted independent ten-row workflow

30457360342 passed all ten rows at 20,099.9 mean tokens per correct answer, \$0.127686 mean estimated cost, 5.7 turns, and 9.8% coefficient of variation. First-pass completion was only 30%: six rows repaired a missing Playwright visibility assertion and one repaired an invalid model-authored pytest assertion. Relative to the favorable five-row estimate, mean tokens rose 11.3%, cost 6.5%, and turns 9.6%. The audit therefore selects `optimize_and_rerun`, not clean promotion. This supplies exact-route, synthetic release-sized evidence while preserving the negative conclusion on real-repository generalization.

Two further trace-derived packages tested whether those repair receipts could be moved into milestone-boundary constraints. Versioned v6 made the existing Playwright visibility criterion explicit and removed the recurring UI repair, but two of five rows still repaired milestone-one Python toolchain mistakes. Versioned v7 additionally made those observed constraints explicit. Its admission and five-row screen passed every row first-pass. The independent ten-row workflow 30547516220 then passed public and hidden validation in all ten rows first-pass at 17,589.9 mean tokens per correct answer, \$0.120940 mean estimated cost, five turns, and 1.62% coefficient of variation. Relative to the v5 ten-row sample, mean tokens fell 12.5%, estimated cost 5.3%, and turns 12.3%, while first-pass completion rose 70 percentage points. This establishes a clean repeated result for the exact synthetic T29, Terra Responses, and FULL controller package; it still does not establish transfer to a fresh real repository.

7 Threats to Validity

Construct validity. GovernanceBench measures repository-task execution under one controller, not general intelligence, developer productivity, code maintainability, or factual truth. Synthetic fixtures improve experimental control but may reward explicit contracts and validators more than naturally occurring repositories.

Internal validity. Proprietary model weights, serving stacks, and sampling implementations are not controlled. Repetition IDs do not guarantee identical provider randomness. Dollar costs are versioned estimates, not invoices. We mitigate these threats with same-commit matched cells, exact model-route labels, raw token receipts, fail-closed provider handling, and paired uncertainty.

External validity. Eight tasks are too few for an unconditional market-wide replacement claim. Task-cluster intervals expose this limitation but cannot manufacture diversity. The benchmark is public and may eventually be contaminated. Replication on fresh, real repositories such as SWE-bench-Live is necessary [17].

Researcher degrees of freedom. Earlier runs informed controller development, so the study includes adaptive optimization. The decisive release profile, thresholds, commit, and selected model pair were fixed before its run. We publish negative admissions and do not pool incompatible commits.

8 Operational Implications

The evidence motivates a guarded router rather than a single default model. Deterministic gates should handle tasks that do not require generation. Lower-tier governed models should receive bounded, validator-rich work for which they have passed admission. Frontier governed models remain appropriate for ambiguous localization, novel architectures, and task families whose lower-tier confidence bound crosses the correctness or efficiency threshold. Routing is therefore based on verified marginal value, consistent with the cost-aware motivation of FrugalGPT and RouteLLM [2, 8], but grounded in executable software evidence.

9 Reproducibility and Ethics

The repository publishes the harness, fixtures, controller, exact workflow inputs, analysis code, compact per-cell measurements, and SHA-256 evidence manifest. Full traces are retained with the cited GitHub Actions runs according to the repository’s configured artifact-retention period; the compact evidence and source digests are versioned permanently. The paper reports model API identifiers but does not infer unpublished parameter counts or architecture. Hidden acceptance tests are isolated from model feedback. Safety tasks reward clarification or refusal rather than destructive execution. No human-subject data are collected.

10 Conclusion

Governance can move work from probabilistic model reasoning into deterministic requirements, retrieval, verification, and stopping. On the evaluated mixed task distribution, this substantially reduced the tokens required for correct software changes and allowed the governed Terra route to clear the preregistered substitution gates against raw Sol. The coding-only gate remained inconclusive by 0.7 percentage points, and the published-size admissions failed. The scientifically defensible result is task-conditional system substitution with explicit confidence and routing gates—not that smaller models universally replace frontier models.

References

- [1] Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nihyporuk, Justin Szeto, Naz Sepah, Edward Raff, et al. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems*, volume 3, 2021. URL <https://arxiv.org/abs/2103.03098>.
- [2] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023. URL <https://arxiv.org/abs/2305.05176>.
- [3] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi: 10.1214/aos/1176344552.
- [4] Hugging Face. Inference endpoints. https://huggingface.co/docs/huggingface_hub/guides/inference_endpoints, 2026. Accessed July 2026.
- [5] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024. URL <https://arxiv.org/abs/2310.06770>.
- [6] Shanchao Liang, Spandan Garg, and Roshanak Zilouchian Moghaddam. The SWE-Bench illusion: When state-of-the-art LLMs remember instead of reason. *arXiv preprint arXiv:2506.12286*, 2025. URL <https://arxiv.org/abs/2506.12286>.
- [7] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36, 2023. URL <https://arxiv.org/abs/2303.17651>.
- [8] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024. URL <https://arxiv.org/abs/2406.18665>.
- [9] Qwen Team. Qwen3-Coder-30B-A3B-Instruct-FP8 model card. <https://huggingface.co/Qwen/Qwen3-Coder-30B-A3B-Instruct-FP8>, 2026. Accessed July 2026.
- [10] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023. URL <https://arxiv.org/abs/2303.11366>.

- [11] Mohammed Latif Siddiq, Arvin Islam-Gomes, Natalie Sekerak, and Joanna C. S. Santos. Large language models for software engineering: A reproducibility crisis. *arXiv preprint arXiv:2512.00651*, 2025. URL <https://arxiv.org/abs/2512.00651>.
- [12] vLLM Project. Tool calling. https://docs.vllm.ai/en/v0.12.0/features/tool_calling/, 2026. Accessed July 2026.
- [13] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.
- [14] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024. URL <https://arxiv.org/abs/2407.01489>.
- [15] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024. URL <https://arxiv.org/abs/2405.15793>.
- [16] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- [17] Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, et al. SWE-bench goes live! *arXiv preprint arXiv:2505.23419*, 2025. URL <https://arxiv.org/abs/2505.23419>.